

Project Report



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Performance Analysis and Modeling of Software Systems (PAMS) - Summer Semester 2026

Dang Minh Nguyen (2809458)
July 13, 2026

Contents

1	Application Description and Design	2
1.1	General Description	2
1.2	Detailed Description of Modules Under Test	3
2	Benchmark Harness Design / Application Changes	4
2.1	Workload	4
2.2	Code Changes for Logging (and Benchmark Client)	5
2.3	Scripts for Running Experiments	6
2.4	Data Collection	6
2.5	Data Processing and Statistics	6
2.6	Plotting Method	6
3	Experiments	7
3.1	Environment	7
3.1.1	Cluster Setup	7
3.1.2	Benchmark Client	7
3.2	Experiment 1: Cache Effectiveness	7
3.2.1	Setup	7
3.2.2	Hypothesis	7
3.2.3	Results and Discussion	8
3.3	Experiment 2: Eviction Policy	9
3.3.1	Setup	9
3.3.2	Hypothesis	9
3.3.3	Results and Discussion	9
3.4	Experiment 3: Increasing number of workers	12
3.4.1	Setup	12
3.4.2	Hypothesis	12
3.4.3	Results and Discussion	12
4	Modeling of the Application	13
4.1	Model Choice	13
4.2	Parameter Estimation	13
4.3	Results	13
4.4	Summary	15

VM virtual machine

MVCC multi-version concurrency control

LRU Least Recently Used

FIFO First In First Out

CLI Command Line Interface

GCP Google Cloud Platform

1 Application Description and Design

toyDB [1] is a distributed SQL database written in Rust and developed primarily as an educational project. It is intended to be simple and understandable, and also functional and correct. Other aspects like performance, scalability, and availability are non-goals [1].

- Raft-based distributed consensus for linearizable state-machine replication.
- ACID transactions with multi-version concurrency control (MVCC) and snapshot isolation.
- A pluggable storage engine with BitCask-style and in-memory backends.
- An iterator-based query engine with simple heuristic optimization and time-travel support.
- A SQL interface supporting basic DDL/DML, joins, aggregates, and transactions.

toyDB was originally written around 2020 as a vehicle to explore database internals and to serve as a didactic counterpart to large-scale distributed SQL systems such as CockroachDB [2].

1.1 General Description

In this project, toyDB is used as the database system and treat as an example of a distributed relational DBMS.

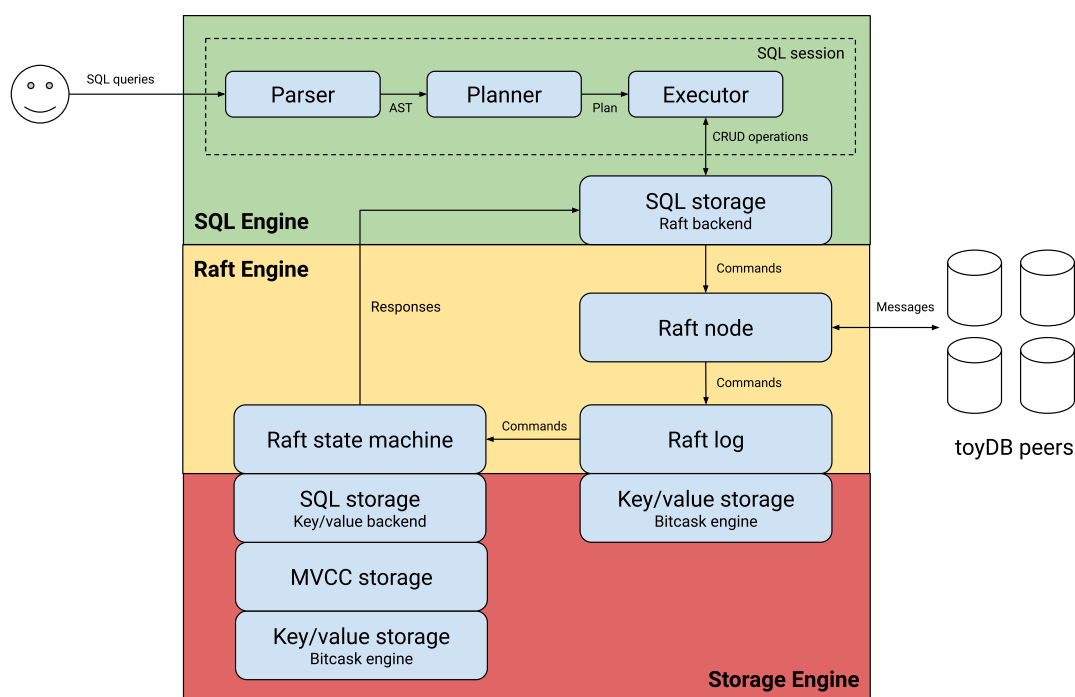


Figure 1: Architecture of toyDB [1]

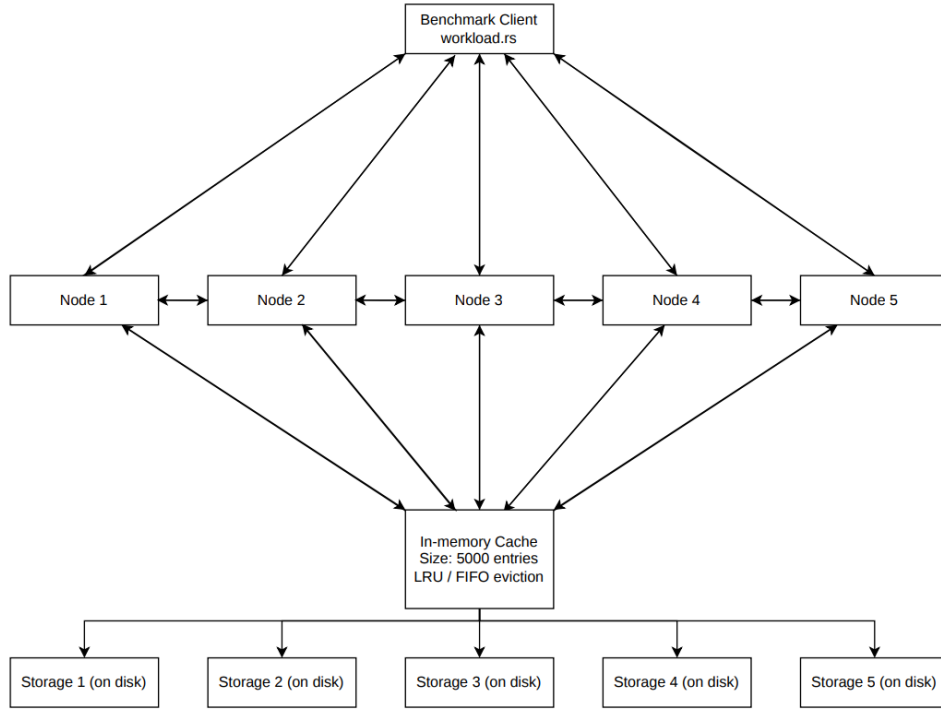


Figure 2: Project Structure

toyDB follows the architecture of a distributed database. Clients communicate with a toyDB node through a SQL interface. Incoming SQL statements are parsed, transformed into a logical query plan by the storage engine, and finally converted into a physical execution plan. By default, a toyDB cluster consists of five nodes.

To improve the performance of read requests, a cache storing key-value pairs is implemented. This project focuses on evaluating and benchmarking the performance of the cache.

1.2 Detailed Description of Modules Under Test

The cache stores the responses to read requests and retrieves data from the database on a cache miss. Two replacement strategies, Least Recently Used (LRU) and First In First Out (FIFO), are evaluated in the experiments to investigate how the frontend cache influences database performance under different dataset sizes and numbers of worker threads.

Instead of implementing a separate cache pre-loading routine, the benchmark prepares the initial dataset during the workload’s **prepare** step. During execution, the workload reports benchmark statistics every second. For the analysis, the time spent creating and initializing the data in the **read** table is considered the preparation phase, while the subsequent steady-state interval is used for comparing different configurations.

In each preparation phase, a random dataset is initialized using the same random seed. After each run, both the cluster data and the cache are cleared to ensure that the next experiment starts from the same initial state. All experiments use the **same cache size of 5 000 entries**. By default for the first and the third experiments, the cache uses **LRU as eviction strategy**.

Experiment	Title	Core Objective
Exp 1	Cache Effectiveness	Quantify how a frontend cache affects throughput, latency percentiles, and the cache hit/miss ratio under different dataset sizes.
Exp 2	LRU vs. FIFO	Compare LRU and FIFO across small and large datasets with both uniform and Zipfian ($\alpha = 1.0$) access patterns.
Exp 3	Increasing Number of Workers	Evaluate cache and database performance under increasing concurrency by varying the number of workers ($K \in \{4, 8, 16, 32, 64\}$).

Table 1: Summary of the experimental framework.

2 Benchmark Harness Design / Application Changes

The benchmark client (`workload.rs`) connects to the cache by overriding the host list using `-H`, without any changes to the client logic. On each request, the database checks an in-memory cache. On a cache hit, it returns the cached result directly. On a miss, it forwards the request to the backend toyDB cluster, caches the response (subject to eviction policy), and returns it to the client. **For all experiments, higher throughput and lower latency indicate better performance.**

To evaluate the performance impact of the caching frontend, I will reuse toyDB's existing workload benchmark client (`src/bin/workload.rs`). The workload tool spawns a configurable number of concurrent worker threads (`-concurrency`) and measures end-to-end transaction latency on the client side using `Instant::now()` and an HDR histogram (`hdrhistogram`) to report throughput and latency percentiles (p50/p90/p99/max).

Latency percentiles (p50/p90/p99/max). In this project, the benchmark is evaluated using a set of metrics **p50, p90, p99, max**.

- *p50* is the median latency: 50% of requests finish within the reported latency.
- *p90* is the 90th percentile: 90% of requests finish within the reported latency.
- *p99* is the 99th percentile: 99% of requests finish within the reported latency.
- *max* is the maximum latency observed so far in the run (worst single transaction).

These higher percentiles characterize *tail latency*, which is often critical for user-perceived performance and can increase under contention, queuing, or outliers. The benchmark is run as a **closed system** in the sense of the lectures: a fixed number of client threads is used, and each thread waits for the response before sending the next request.

2.1 Workload

Each experiment is repeated five times, and the results are aggregated for evaluation. The workload consists of point reads over a pre-populated dataset of key-value pairs at two sizes:

- Small (s): 1,000 rows
- Large (l): 10,000 rows

The access pattern follow one of two distributions:

- Uniform: every key is equally likely to be accessed.
- Zipfian: a small number of hot keys receive most of the requests, modelling the skewed access pattern.

The workload flow is as follow:

- **Workload generation.** After the initial bulk insertion of all keys, the benchmark begins. A `BlockGen` iterator shuffles all key IDs into a fresh list and interleaves requests in blocks of 100 operations:
 - **Fresh blocks** draw keys exclusively from the pool of previously untouched keys. Each key is accessed for the first time, resulting in a cache miss and insertion into the cache.
 - **Used blocks** draw exclusively from keys that have already been accessed. Keys are selected either uniformly at random or according to a Zipfian distribution ($\alpha = 1.0$) to model real-world access locality.

Fresh and used blocks alternate throughout the run until every key has been touched at least once; after that point all blocks read from the full key set. Each request operates on a single key (batch size = 1).

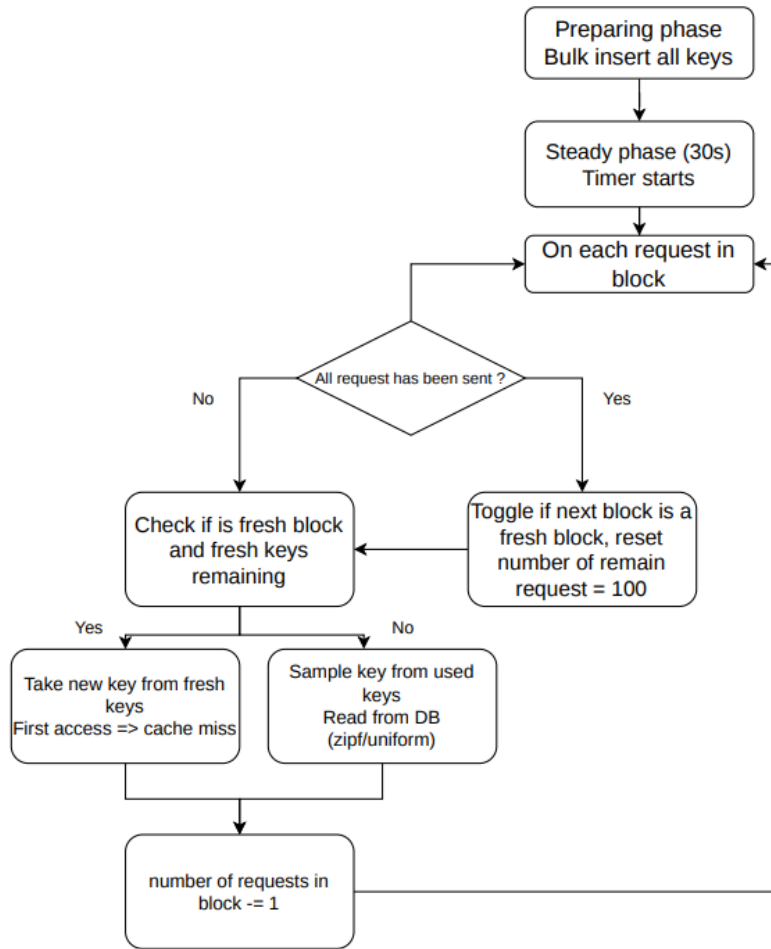


Figure 3: Workflow of the benchmark.

The query access pattern alternates in blocks of 100 operations: a fresh block reads 100 previously unseen keys (incurring a cache miss on first access), followed by a used block that samples 100 keys from the growing pool of already-accessed keys (following a Zipfian or uniform distribution). This alternation repeats until every key has been touched at least once; from that point onward all operations draw from the full key set. The benchmark runs for 30 seconds before terminating.

2.2 Code Changes for Logging (and Benchmark Client)

In the benchmark client (`workload.rs`), each worker measures the latency of every request using `Instant`. Every second, the benchmark reports throughput (transactions/s) together with latency percentiles (p50, p90, p99, and max). This ensures that the measured metrics are end-to-end and comparable across different configurations.

To further analyze cache behavior, additional counters are introduced to record cache hits and misses. These statistics are reported together with the benchmark results, allowing the cache hit ratio to be evaluated alongside the performance metrics.

For the experiments, the benchmark client is also extended to automatically export the results of each benchmark run as CSV files without requiring manual file paths. In addition, a Command Line Interface (CLI) is added to `workload.rs` to configure benchmark parameters such as the dataset size, eviction strategy, and number of worker threads.

- `-experiment`: an experiment tag used in output filenames.
- `-out-dir`: output directory for benchmark artifacts (defaults to `csv/`).
- At startup, the client ensures the output directory exists via `create_dir_all(out_dir)` and generates a unique `run_id` from `SystemTime` (milliseconds since `UNIX_EPOCH`).
- It then writes per-second measurements to `<out-dir>/<experiment>-<run_id>.csv` and writes the final one-row summary to `<out-dir>/<experiment>-<run_id>-summary.csv`.

2.3 Scripts for Running Experiments

Benchmark runs are executed via experiment scripts, which standardize the parameter sets (dataset size, concurrency, whether the cache is enabled, and which eviction policy is used) and thus ensure consistent data collection across repetitions. After each run, the generated CSV files are collected directly from `csv/` and used as input for subsequent data processing and plotting. Every combination of factors is repeated five times (`-id 1..5`) to obtain statistically meaningful estimates.

All experiment scripts reside in `scripts/cloud/` following a uniform directory hierarchy:

```
scripts/
-- cloud/
  |-- exp1/
  |   -- {factor}/{size}/{dist}.sh
  |-- exp2/
  |   -- {factor}/{size}/{dist}.sh
  -- exp3/
     -- {factor}/{size}/{dist}.sh
```

Each script accepts an optional numeric ID argument (default: 1) that controls the repetition index.

Sometimes, when the cluster encounters errors after running for a long time, the `sanitize-cloud.sh` script is used to delete `/opt/toydb/data/raft` and `/opt/toydb/data/sql` on all nodes and restart the `toydb` service, ensuring a clean starting state.

2.4 Data Collection

Each benchmark execution produces two files:

- a **per-second time series** CSV containing `time_s`, `progress`, `txns`, `throughput`, `p50_ms`, `p90_ms`, `p99_ms`, `max`, `cache_hits`, `cache_misses`, `cache_hit_rate`, and
- a **final one-row summary** CSV containing all run configuration and results: `experiment`, `run_id`, `workload`, `hosts`, `concurrency`, `count`, `seed`, `total_time_s`, `txns`, `throughput`, `p50_ms`, `p90_ms`, `p99_ms`, `max`, `cache_hits`, `cache_misses`, `cache_hit_rate`, `duration`.

Filenames include the `-experiment` tag and a millisecond Unix timestamp as `run_id` to prevent overwriting and to enable grouping runs by configuration.

Benchmark runs are executed via experiment scripts (one per experiment), which standardize the parameter sets (dataset size, concurrency, and whether the cache is enabled and which eviction policy is used) as mentioned in Section 2.3

2.5 Data Processing and Statistics

Each benchmark run produces two CSV files: a per-second time-series file and a one-row summary file containing the configuration metadata. Because the benchmark only captures the steady-state phase, the raw data requires no trimming or discarding.

To aggregate results across repetitions, `compute-mean.py` groups the time-series rows by `time_s` and computes the mean alongside a 95% confidence interval for each numeric column. Since each configuration is repeated only five times ($n = 5$), the interval uses the t -distribution with 4 degrees of freedom ($t_{0.025}(4) = 3.182$) instead of the normal approximation. The output is written to `avg.csv`.

A separate script, `compute-exp3-avg.py`, performs the same aggregation but merges all five concurrency levels (c4, c8, c16, c32, c64) into a single `avg-exp3.csv` for Experiment 3.

The 11 time-series columns include throughput, progress, latency percentiles (p50, p90, p99, max), and cache statistics (hits, misses, hit rate). Plotting scripts read either the per-run CSVs or the pre-computed `avg.csv` depending on the chart type.

2.6 Plotting Method

Since the benchmark records only the steady phase, the Python scripts using `matplotlib` plot the raw `.csv` output files directly without any discarding or trimming. Each experiment is repeated five times; results are aggregated and displayed with a 95% confidence interval.

Bar charts are used to visualize latency figures, while line charts are used to visualize throughput.

The M/M/m model is implemented following lecture 5a (Queueing Theory): given the measured arrival rate λ and the estimated service rate $\mu = 1/E[s]$ estimated from the lowest concurrency measurement ($K = 4$) using Little's law, the model predicts the mean response time $E[r]$ and the saturation throughput using the closed-form queueing formulas for ρ , p_0 , q , and $E[r]$.

Common plotting infrastructure is declared in `config.py`, which defines the color palette, grid style, CI computation helpers (using the t -distribution for small samples), and legend placement. All scripts import from this module to ensure visual consistency across figures.

3 Experiments

3.1 Environment

3.1.1 Cluster Setup

The cluster of toyDB was deployed on **Google Cloud Platform (GCP)** using **Terraform**. Each node runs on a separate **e2-medium** virtual machine (VM) (2 vCPU, 4GB RAM) with Ubuntu 22.04. Each node of the cluster was run on a separate VM and they run **on the same zone (europe-west3-c)**.

3.1.2 Benchmark Client

The benchmark client is a Rust binary (`workload.rs`) built with `cargo run --release` and executed on the local development machine. It connects to the cluster by providing the five GCP VM addresses via the `-H` flag (or the `$TOYDB_HOSTS` environment variable). Each run is launched through the corresponding experiment script under `scripts/cloud/`, which standardises all parameters. By default, the client spawns 16 concurrent workers (`-concurrency 16`) and runs point-read queries for 30 seconds.

3.2 Experiment 1: Cache Effectiveness

This experiment measures the impact of the in-memory caching on throughput and latency. By comparing configurations with and without the cache across small (1 000 rows) and large (10 000 rows) datasets, the performance gains from memory-resident data versus the cost of backend round-trips are quantified.

Purpose: The goal of this experiment is to quantify how an in-memory caching frontend changes toyDB's throughput and latency. By comparing small and large datasets, with and without the cache, it is aimed to understand how cache hit ratio and dataset size influence performance.

3.2.1 Setup

The experiment follows a $2 \times 2 \times 2$ factorial design:

- **Cache:** enabled (`-cache`) or disabled.
- **Dataset size:** 1 000 rows (small) or 10 000 rows (large).
- **Access distribution:** uniform or Zipfian ($\alpha = 1.0$).

Each combination is repeated five times. The client spawns 16 concurrent workers and runs for 30 seconds. The cache stores 5 000 entries using LRU eviction (default). The cluster consists of five GCP **e2-medium** nodes; the client connects via `-H $TOYDB_HOSTS`.

3.2.2 Hypothesis

- **Cache vs. no cache:** Enabling the cache will increase throughput and reduce latency percentiles (p50, p90, p99, max) by serving repeated reads from memory instead of forwarding them to the backend cluster.
 - **Uniform:** The small dataset fits entirely in the cache. After the first few misses, almost every request is a hit. With the large dataset, only half the keys fit in the cache, so the hit ratio stops at around 50%.
 - **Zipfian:** A few hot keys get most of the requests. LRU keeps these hot keys in the cache, so the hit ratio stays high even with the large dataset. This means bigger performance gains than with uniform access.
- **Effect of dataset size:** The relative performance benefit of the cache will be larger for the small dataset than for the large dataset, because the small dataset fits entirely in the cache (near-100% hit rate). With the large dataset, at most half the keys fit, so only about half of the requests avoid a backend round-trip.
- **Hit ratio correlation:** Across all runs, configurations with a higher cache hit ratio will show higher throughput and lower latency percentiles than configurations with a lower hit ratio.

3.2.3 Results and Discussion

Figure 4 compares the throughput of the cache-enabled and cache-disabled configurations for the large dataset under different access patterns. The cache improves throughput by approximately $3\times$ (from ~ 831 txns/s to $\sim 3\,025$ txns/s), matching the first hypothesis. The confidence bands are narrow, indicating low run-to-run variability.

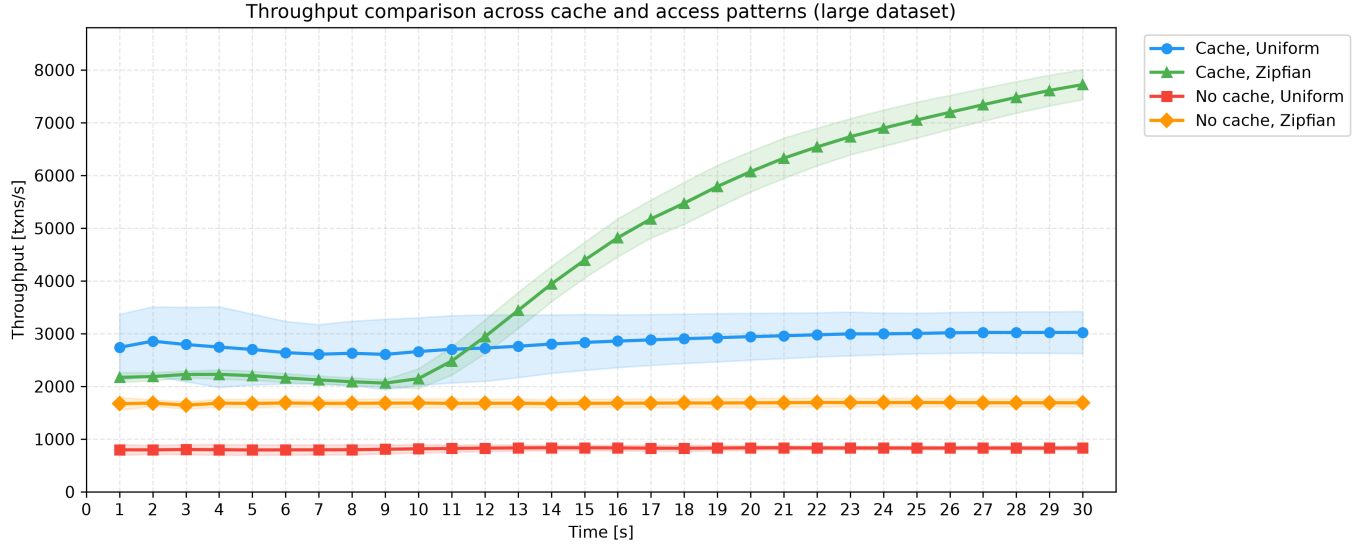


Figure 4: Throughput over time for the large dataset (10 000 rows) under uniform and Zipfian access, with and without the cache. Shaded bands show the 95% confidence interval.

Under Zipfian access, the cache gain is even larger. Because hot keys remain resident under LRU, the hit ratio stays high and throughput reaches up to $\sim 7\,724$ txns/s, roughly $5\times$ the no-cache baseline. This supports the third hypothesis: skewed workloads benefit more from caching than uniform workloads.

Additionally, Figure 4 compares the throughput of Zipfian and uniform access (both with the cache on). Zipfian is faster because hot keys stay in the cache, giving a higher hit rate. Uniform access spreads requests evenly, so many keys get evicted before they are requested again. This supports the hypothesis that a skewed workload benefits more from caching.

Figures 5 shows the hit ratios for the small dataset under Zipfian access pattern. The distribution reaches near-100% hit rates, but Zipfian access stabilises faster because hot keys are retained by LRU. With the small dataset the entire working set fits in the cache (1 000 rows $<$ 5 000 cache entries), so the dataset-size hypothesis is confirmed: the small dataset benefits more from caching than the large dataset.

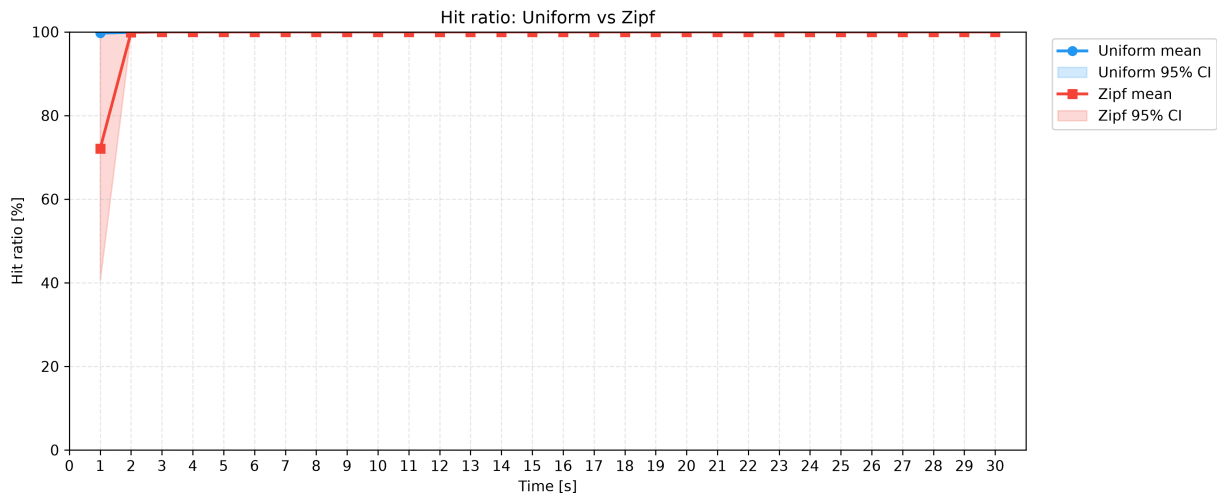


Figure 5: Hit ratio for small dataset: uniform vs. Zipfian access.

3.3 Experiment 2: Eviction Policy

Experiment 2: Eviction Policy. In this experiment, the efficiency of different eviction strategies (LRU vs FIFO) is evaluated for a range of cache sizes and key ranges. The Performance across small and large dataset size and across uniform and Zipfian access pattern, to understand when a more sophisticated eviction policy (LRU) provides a clear benefit over a simpler one (FIFO).

Purpose: The goal of this experiment is to quantify how eviction policy affects throughput, latency, and cache hit ratio. By comparing runs with the same workload but different policies, I aim to identify: (i) the regimes where LRU significantly outperforms FIFO, (ii) where both behave similarly, and (iii) how sensitive the system is to cache sizing errors (too small vs. comfortably large caches).

3.3.1 Setup

As in the other experiments, this experiment is conducted using two dataset sizes: *small* and *large*. The cache size is fixed at the default value of 5 000 entries. In this experiment, the cache uses the FIFO eviction policy.

For the LRU policy, the results obtained in Experiment 1 (where caching is enabled with the default LRU policy) are reused and compared with the corresponding FIFO results.

3.3.2 Hypothesis

- **Working set fits in cache:** When the working set fits entirely in the cache, both LRU and FIFO reach a 100% hit rate and converge to the same hardware-limited throughput during steady state. Any differences between the two policies are limited to the warm-up phase (<1 second) and is negligible for the 30-second measurement window. This case therefore serves as a baseline confirming that the caching overhead does not degrade maximum throughput.
- **Working set larger than cache with temporal locality** When the key range is larger than the cache but the workload exhibits temporal locality (recently accessed keys are likely to be accessed again), LRU is expected to achieve a higher hit ratio than FIFO. As a result, LRU should provide higher throughput and lower latency percentiles than FIFO, despite its slightly higher bookkeeping overhead.
- **Working set larger than cache with weak locality** When the number of keys are larger than the cache and access patterns are uniform, LRU and FIFO are expected to achieve similarly low hit ratios. In this regime, the simpler FIFO policy may show better latency than LRU because it incurs less per-operation overhead while offering no worse hit ratio.

3.3.3 Results and Discussion

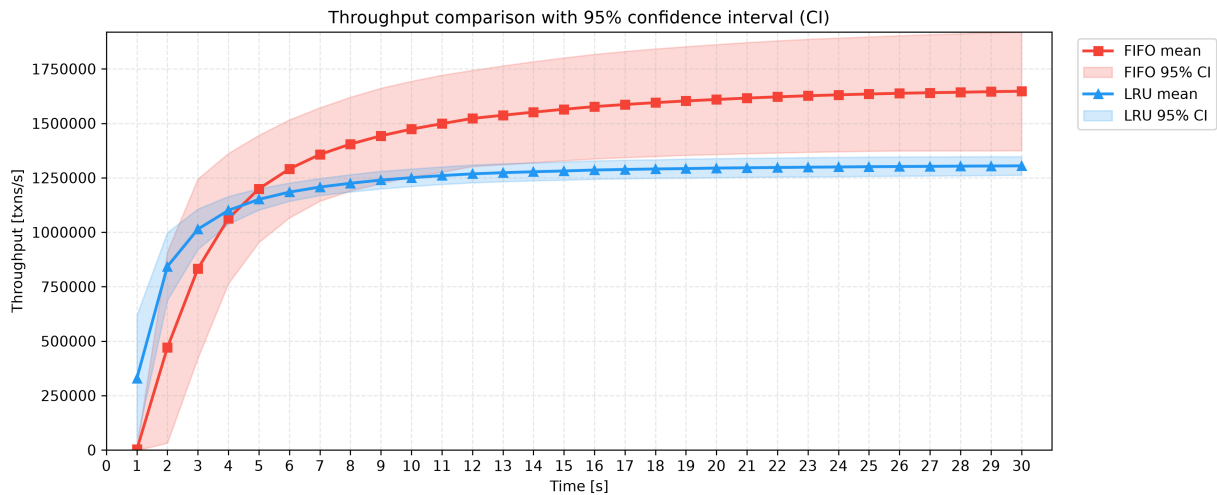


Figure 6: Throughput comparison between LRU and FIFO on small dataset size, uniform access pattern.

Figure 6 compares the throughput of LRU and FIFO for the small dataset under uniform access. Both policies reach a 100% hit rate after the first two seconds and converge to the same maximum throughput of roughly 1.2 million txns/s. However, afterwards, the FIFO throughput reaches higher, about 1.7 million txns/s.

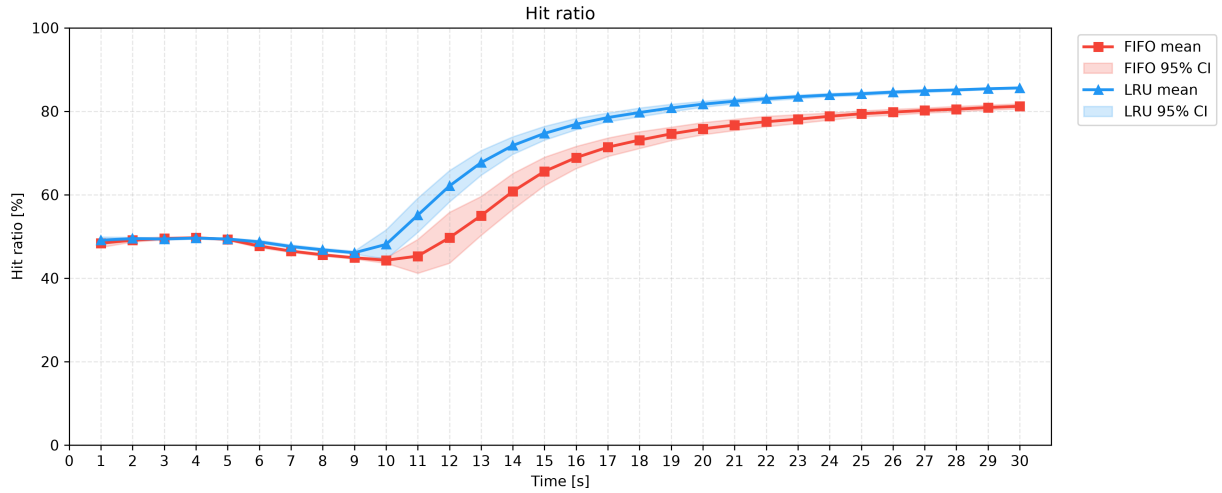


Figure 7: Hit ratio comparison between LRU and FIFO on large dataset size, Zipfian access pattern.

The warm-up phase is too short to affect the 30-second measurement. This confirms that when the working set fits, eviction policy has no impact on steady-state performance.

With the large dataset and Zipfian access, the eviction policy makes a clear difference. Figure 7 shows that LRU maintains a higher hit ratio (85.6%) than FIFO (81.2%). Hot keys stay in the LRU cache because they are accessed frequently, but FIFO evicts them in insertion order regardless of popularity.

As a result, Figure 8 shows that LRU delivers roughly 43% higher throughput (7 724 vs. 5 392 txns/s). This matches the hypothesis: LRU outperforms FIFO when the workload has access skew.

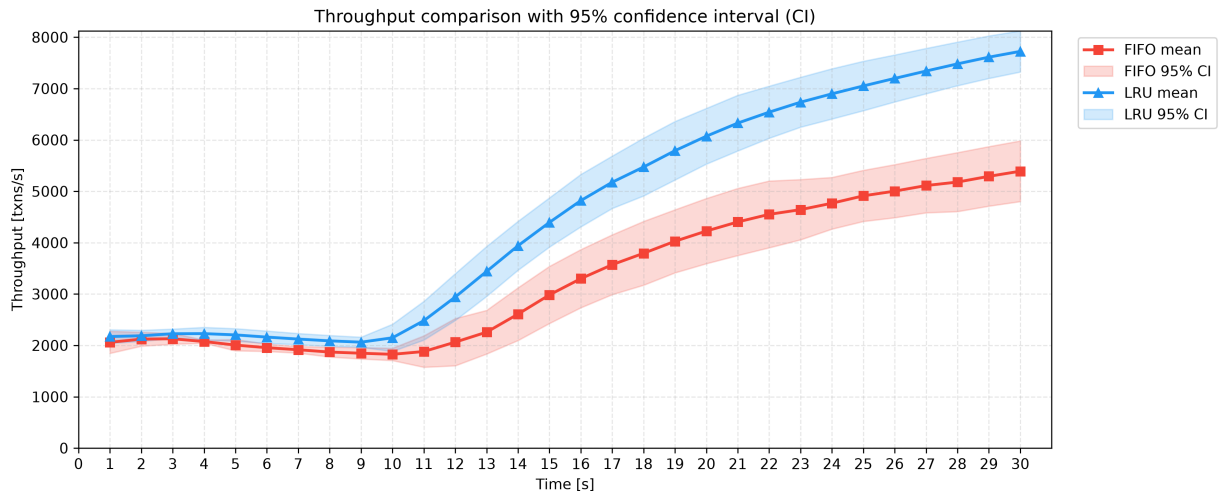


Figure 8: Throughput comparison between LRU and FIFO on large dataset size, Zipfian access pattern.

Under uniform access, both policies perform poorly because every key has the same chance of being requested. Figure 9 shows throughput of only 2,000–3,000 txns/s for both LRU and FIFO, far below the small-dataset peak. The hit ratios are similarly low (48%).

Figure 10 compares the tail latency. FIFO shows slightly lower latency than LRU, despite having a similar hit ratio. This matches the hypothesis: FIFO avoids the overhead of reordering entries on every cache hit, which helps when both policies deliver similarly low hit ratios.

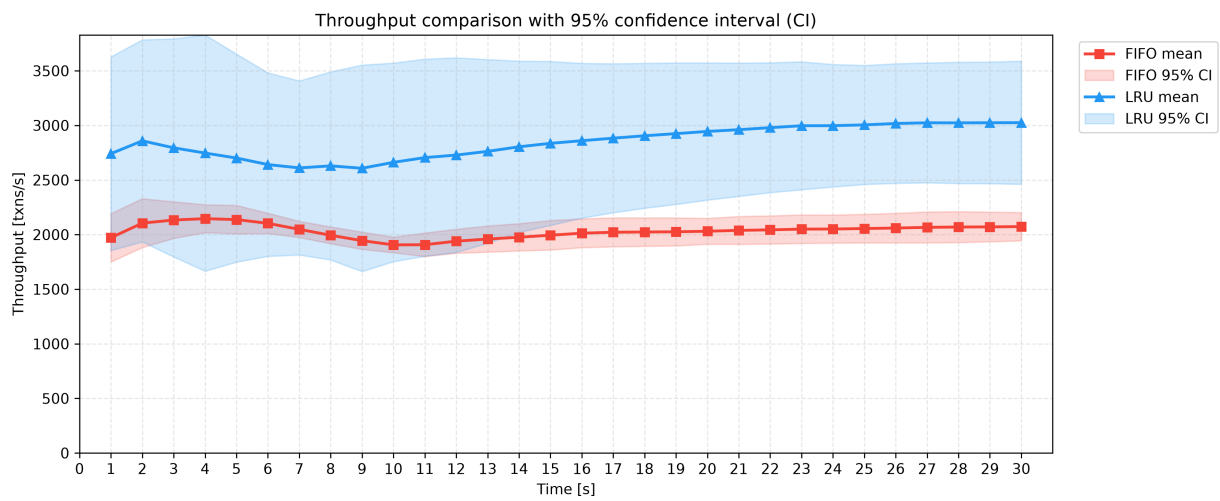


Figure 9: Throughput comparison between LRU and FIFO on large dataset size, uniform access pattern.

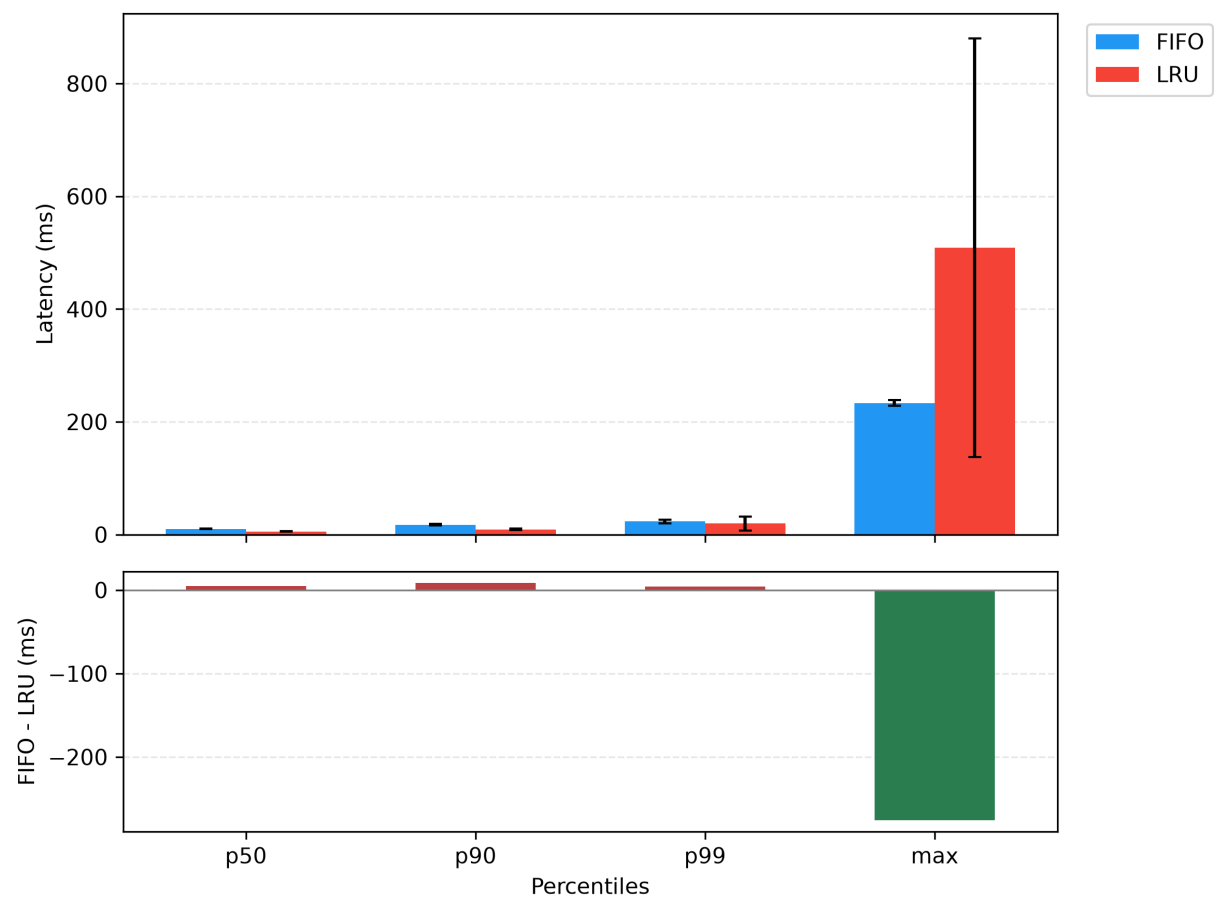


Figure 10: Latency comparison between LRU and FIFO on large dataset size, uniform access pattern.

3.4 Experiment 3: Increasing number of workers

Increase the number of workers with different concurrency levels $K \in \{4, 8, 16, 32, 64\}$ to observe how the cache and the database perform under increasing load. A single work generator produces a stream of key lookups, and all workers consume from the same queue. Each request goes to exactly one worker — no two workers execute the same query.

3.4.1 Setup

Same as other experiments, the cache capacity and policy are selected as default (5000 entries with LRU). The experiment was performed on two dataset size small and large under two access pattern uniform and Zipfian

3.4.2 Hypothesis

- As the number of concurrent workers K increases, throughput should increase at first because more requests are processed in parallel. Eventually the cluster becomes the bottleneck and throughput stops growing. The small dataset (more cache hits) scales further than the large dataset before reaching this limit.
- When K becomes 8 for the chosen cache capacity and number of database workers, it is expected to see signs of saturation (e.g., response time growing quickly and throughput increasing more slowly or even flattening).
- Cache hit ratio is independent of K : Because all workers share the same cache and draw keys from the same stream, the cache hit ratio does not change with K . The benefit of caching over no-cache therefore persists at all concurrency levels.

3.4.3 Results and Discussion

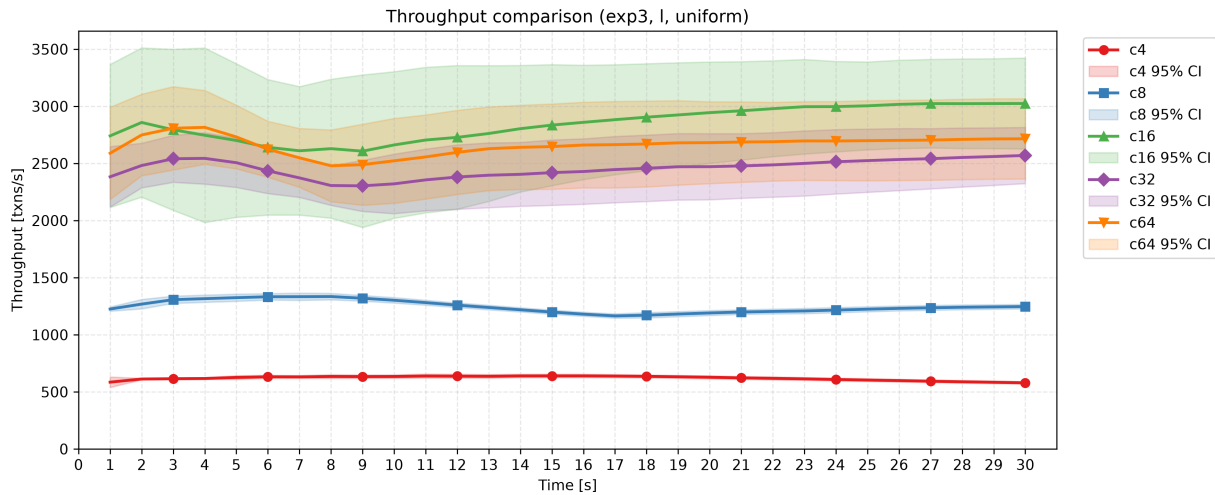


Figure 11: Throughput over time for the large dataset (10 000 rows) under uniform access at different concurrency levels K . Each line represents one worker count, with shaded bands showing the 95% confidence interval.

For the large dataset under uniform access, throughput grows from 580 txns/s at $K=4$ to 3 000 txns/s at $K=16$, then slightly degrades at higher concurrency due to contention. Adding more workers beyond $K=16$ does not improve throughput because the backend cluster (5 nodes) is already fully utilised — most requests miss the cache and hit the database. This confirms the second hypothesis: saturation appears early when the working set exceeds the cache capacity.

The small dataset behaves differently. Throughput stays flat at roughly 1.3–1.4 million txns/s across all concurrency levels. Because the entire dataset fits in the cache, workers rarely touch the backend, so the bottleneck becomes lock contention in the shared cache. Throughput is flat at all K because the cache mutex limits concurrency, not the backend.

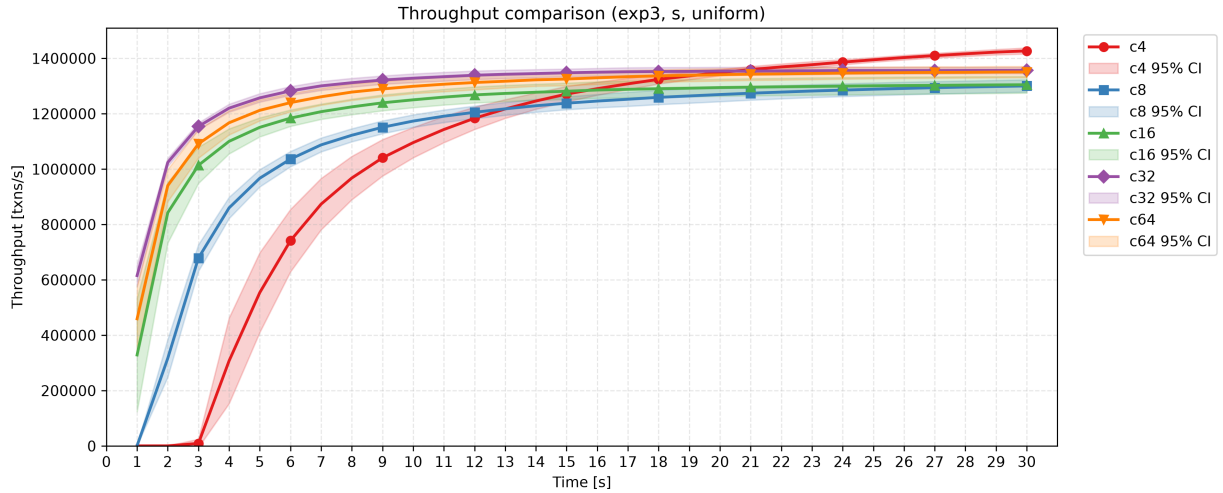


Figure 12: Throughput over time for the small dataset (1 000 rows) under uniform access at different concurrency levels K . Each line represents one worker count, with shaded bands showing the 95% confidence interval.

4 Modeling of the Application

This section models Experiment 3 using the M/M/m queueing model from lecture 5a. The goal is to predict how throughput and response time scale with the number of concurrent workers K , and to compare the predictions against the measured results.

4.1 Model Choice

The system is modelled as a closed M/M/m queue: K concurrent workers act as m parallel servers (workers), each processing one request at a time. When a worker sends a request, it waits for the response before sending the next one. The service rate μ captures the time a backend node takes to handle a single request. →**This model fits the large dataset under uniform access, where most requests miss the cache and reach the database cluster.**

4.2 Parameter Estimation

The arrival rate λ is the measured throughput from Experiment 3. The service rate $\mu = 1/E[s]$ is estimated from the lowest concurrency measurement ($K = 4$), where queueing is negligible. At $K = 4$, the observed response time approximates the service time: $E[s] \approx E[r]$. By Little's law, $E[r] = K/\lambda(4) = 4/\lambda(4)$. Substituting gives

$$\mu = \frac{1}{E[s]} \approx \frac{1}{E[r]} = \frac{1}{4/\lambda(4)} = \frac{\lambda(4)}{4}.$$

The predicted throughput is computed by solving the closed-system M/M/m fixed-point equation $\lambda = m/E[r](\lambda)$ numerically. The mean response time is

$$E[r] = \frac{1}{\mu} \left(1 + \frac{q}{m(1-\rho)} \right),$$

where $\rho = \lambda/(m \cdot \mu)$, p_0 is the idle probability, and $q = P(\text{queueing})$ (see lecture 5a).

4.3 Results

→**The model is in the right ballpark at low K (4 – 8) but overestimates at high K because M/M/m assumes stable servers, while the real system saturates after $K = 16$ (throughput stays around 2700 txns/s).**

For the Zipfian workload (Figure 14), the M/M/m model predicts much lower throughput than measured (up to 3.8×). This is because most requests hit the cache and never reach the backend, but the model only knows about the backend. The measured throughput plateaus at $K = 32$ due to cache-frontend contention, while the model plateaus because the backend is saturated. The model serves as a lower bound for a system without caching.

With the small dataset (Figure 15), the model completely fails — it predicts linear scaling, but measured throughput is flat (1.3M) at all K because the cache mutex contention is the bottleneck, not the backend.

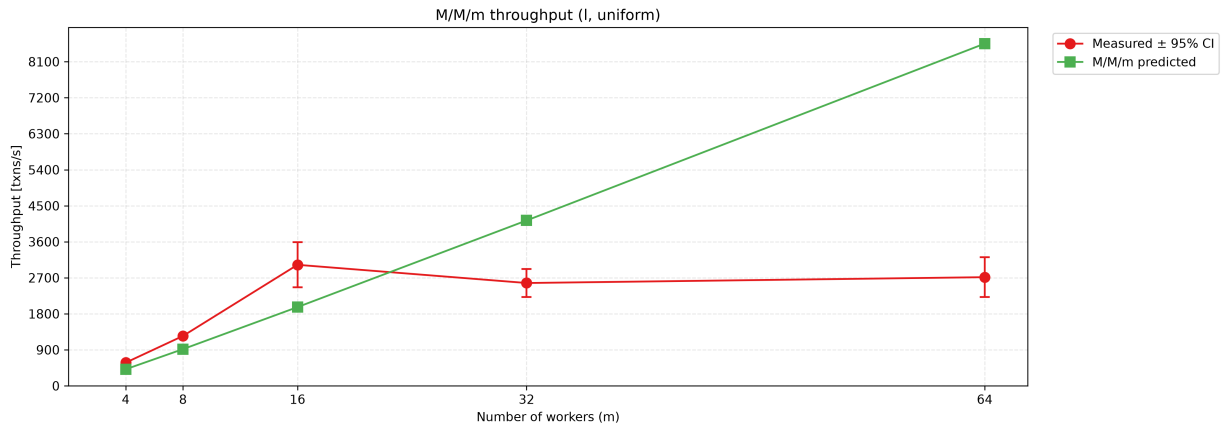


Figure 13: M/M/m throughput prediction versus measured throughput for the large dataset under uniform access. →**The model is in the right ballpark at low K but overpredicts at high concurrency.**

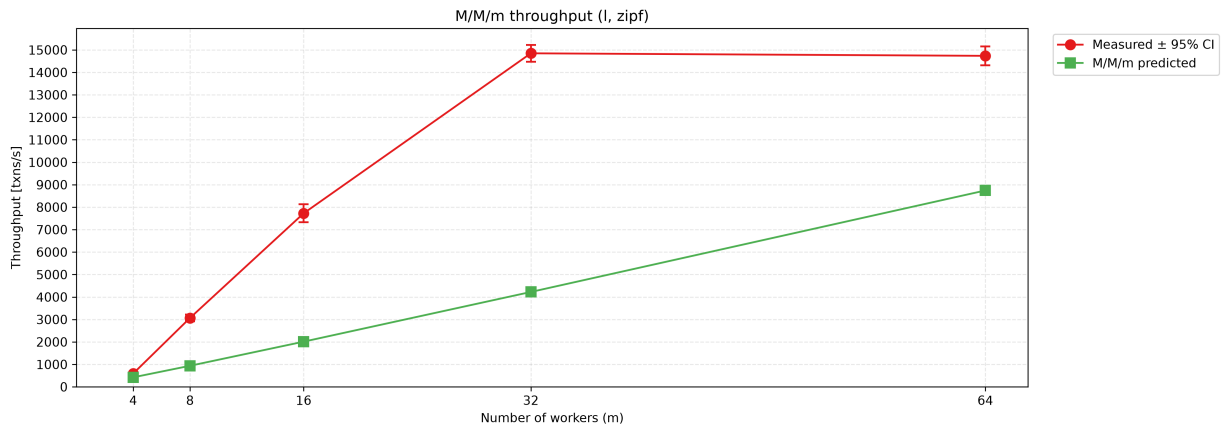


Figure 14: M/M/m throughput prediction versus measured throughput for the large dataset under Zipfian access. The model underestimates throughput because the cache absorbs many requests that the model assumes go to the backend.

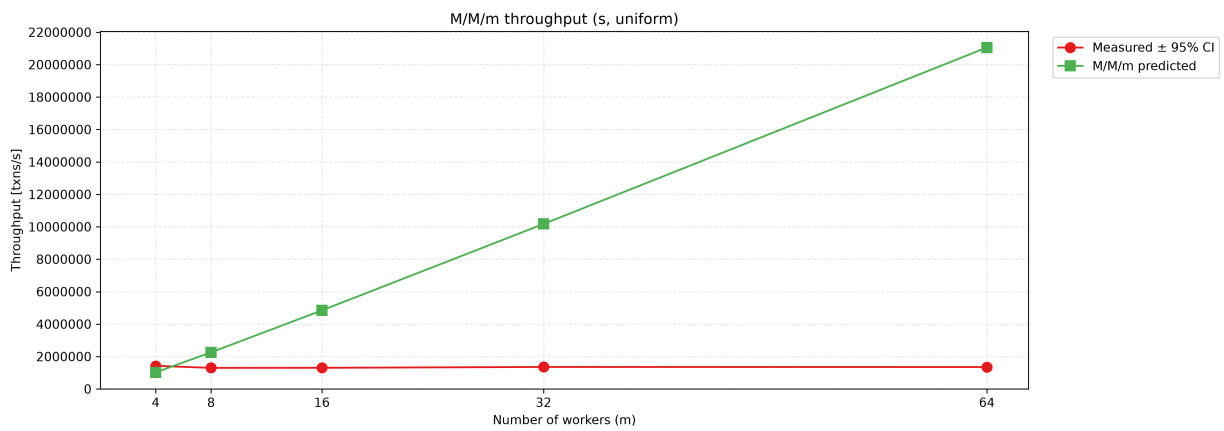


Figure 15: M/M/m throughput prediction versus measured throughput for the small dataset under uniform access. The model significantly overestimates throughput because the cache frontend, not the database, is the real bottleneck.

4.4 Summary

The M/M/m model gives reasonable predictions at low concurrency but deviates at high concurrency, where real-world contention is not captured by the model.** It performs worse when a cache layer shields the backend (Zipfian, small dataset), because the model assumes all m servers face the full workload. For systems with a cache frontend, a two-layer queueing model would be needed for better accuracy.

References

- [1] Erik Grinaker. Toydb: A distributed sql database in rust. <https://github.com/erikgrinaker/toydb>, 2020. Accessed: 2026-05-01.
- [2] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, et al. Cockroachdb: The resilient geo-distributed sql database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1493–1509, 2020. Accessed: 2026-05-10.